

ShopNest DevOps Project

Complete End-to-End Hands-On Guide

PHASE 1 — PROJECT DESIGN

Step-by-step from scratch

DevOps Stack: React · Node.js · PostgreSQL · Docker · Jenkins · Terraform · Ansible · Kubernetes · Prometheus · Grafana

Cloud: AWS | Version: 1.0 | Level: Beginner → Advanced

SECTION 1 — INTRODUCTION & PROJECT OVERVIEW

What Are We Building?

We are building ShopNest — a simplified but realistic e-commerce web application — and deploying it end-to-end using industry-grade DevOps tools and practices. This is not a toy project. Every tool, every config file, and every command in this guide mirrors what real DevOps engineers do at production companies.

i Why E-commerce for DevOps Practice?

- Clear 3-tier split — Frontend, API, Database each have their own container and server
- Realistic traffic patterns — product pages get browsed constantly; checkout spikes during sales
- Stateful data — orders and inventory need reliable database writes, not just reads
- Session management — shopping carts teach Redis caching and connection pooling
- Enough complexity to stress-test every DevOps tool without drowning in app code

Full Technology Stack

Here is every tool we will use across all 15 phases of this project:

Category	Tool / Technology	Purpose	Phase Used
Frontend	React 18 + Nginx	User interface & reverse proxy	Phase 1–4
Backend	Node.js + Express	REST API server	Phase 1–4
Database	PostgreSQL 15	Persistent data storage	Phase 1–4
Cache	Redis 7	Session & cart caching	Phase 2–4
Containers	Docker + Docker Compose	Package and run each tier	Phase 4
CI/CD	Jenkins	Automated build & deploy pipeline	Phase 5
IaC	Terraform	Provision all AWS infrastructure	Phase 6
Config Mgmt	Ansible	Configure servers after creation	Phase 7
Orchestration	Kubernetes (EKS)	Run containers at scale	Phase 8
Package Manager	Helm	Manage Kubernetes deployments	Phase 9
Monitoring	Prometheus + Grafana	Metrics, dashboards, alerts	Phase 10

Logging	Loki + Grafana	Centralised log aggregation	Phase 11
Security	Trivy + SonarQube	Image & code vulnerability scans	Phase 12
Cloud	AWS	All infrastructure hosting	All Phases

SECTION 2 — PREREQUISITES & ACCOUNT SETUP

What You Need Before Starting

Complete every item in this section before moving to Phase 1. Skipping setup is the #1 reason beginners get stuck on Step 3 of Phase 4.

► 2.1 — Create an AWS Account

STEP 1 Go to AWS sign-up page

Action	Open your web browser
URL	<code>https://aws.amazon.com</code>
Click	Create an AWS Account (top right button)

Fill in the form with your email, password, and account name. Choose Personal for account type. Enter your credit/debit card — AWS requires this but we will stay within free tier limits.

STEP 2 Enable MFA on your root account

Login	Sign in to <code>https://console.aws.amazon.com</code>
Click	Your account name (top right) → Security credentials
Scroll	Down to Multi-factor authentication (MFA)
Click	Assign MFA device
Select	Authenticator app → Continue
Action	Scan QR code with Google Authenticator or Authy
Type	Two consecutive 6-digit codes from your app
Click	Add MFA

❑ Security Rule

NEVER use your root account for daily work. Root is only for billing and account-level settings. We will create an IAM user in the next step for all DevOps operations.

STEP 3 Create an IAM admin user

Navigate	AWS Console → Services → IAM
Click	Users → Create user

Type	Username: devops-admin
Check	Provide user access to the AWS Management Console
Select	I want to create an IAM user
Password	Choose: Custom password → type a strong password
Uncheck	Users must create a new password at next sign-in
Click	Next
Select	Attach policies directly
Search	AdministratorAccess → tick the checkbox
Click	Next → Create user
SAVE	Download the CSV with access key details

STEP 4 Create IAM access keys for CLI

Navigate	IAM → Users → devops-admin → Security credentials tab
Click	Create access key
Select	Command Line Interface (CLI)
Tick	I understand the above recommendation...
Click	Next → Create access key
SAVE	Copy Access key ID and Secret access key – you cannot view them again!

► 2.2 — Install Required Software (Your Local Machine)

Install the following on your laptop/desktop. Commands shown are for Ubuntu/Debian Linux. For Windows use WSL2 (Windows Subsystem for Linux) — treat WSL2 as your Linux terminal for this entire guide.

STEP 5 Install AWS CLI

Open a terminal and run these commands one by one:

```
# Download the AWS CLI installer
curl -s https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip -o awscliv2.zip

# Unzip the installer
unzip awscliv2.zip

# Run the install script
sudo ./aws/install
```

```
# Verify the installation
aws --version
# Expected output: aws-cli/2.x.x Python/3.x.x ...
```

Now configure it with the access keys you saved in Step 4:

```
aws configure

# When prompted, type each value and press Enter:
AWS Access Key ID [None]: PASTE_YOUR_ACCESS_KEY_HERE
AWS Secret Access Key [None]: PASTE_YOUR_SECRET_KEY_HERE
Default region name [None]: ap-south-1
Default output format [None]: json
```

Test that it works:

```
aws sts get-caller-identity

# You should see output like this:
# {
#   "UserId": "AIXXXXXXXXXXXXXXXXXX",
#   "Account": "123456789012",
#   "Arn": "arn:aws:iam::123456789012:user/devops-admin"
# }
```

STEP 6 Install Docker

```
# Remove any old Docker versions
sudo apt-get remove docker docker-engine docker.io containerd runc

# Update package index and install dependencies
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin

# Add your user to the docker group (so you don't need sudo every time)
sudo usermod -aG docker $USER
```

```
# Apply group change (or log out and back in)
newgrp docker

# Verify Docker works
docker --version
docker run hello-world
```

STEP 7 Install Node.js (for frontend + backend development)

```
# Install nvm (Node Version Manager) — best practice
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash

# Reload your shell profile
source ~/.bashrc

# Install Node.js LTS (Long Term Support)
nvm install --lts
nvm use --lts

# Verify installation
node --version      # Should show v20.x.x or higher
npm --version       # Should show 10.x.x or higher
```

STEP 8 Install Git

```
sudo apt-get install -y git

# Set your identity (used in every commit message)
git config --global user.name "Your Full Name"
git config --global user.email "your@email.com"

# Set default branch name to main
git config --global init.defaultBranch main

# Verify
git --version
```

STEP 9 Install Terraform

```
# Add HashiCorp GPG key and repository
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt-get update && sudo apt-get install -y terraform

# Verify
terraform --version
```

STEP 10 Install kubectl

```
# Download the latest stable release
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

# Install it
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Verify
kubectl version --client
```

STEP 11 Install Helm

```
curl https://baltocdn.com/helm/signing.asc | gpg --dearmor | sudo tee
/usr/share/keyrings/helm.gpg > /dev/null

echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/helm.gpg] \
https://baltocdn.com/helm/stable/debian/ all main" | \
sudo tee /etc/apt/sources.list.d/helm-stable-debian.list

sudo apt-get update && sudo apt-get install -y helm

# Verify
helm version
```

✓ Installation Checklist — Tick Off Each Tool

- ☐ AWS CLI installed → `aws --version`
- ☐ AWS configured → `aws sts get-caller-identity`
- ☐ Docker installed → `docker run hello-world`
- ☐ Node.js installed → `node --version`
- ☐ Git installed → `git --version`
- ☐ Terraform installed → `terraform --version`
- ☐ kubectl installed → `kubectl version --client`
- ☐ Helm installed → `helm version`

PHASE 1 — PROJECT DESIGN

In this phase you will not write any application code. You are the architect. Your job is to design the entire system before a single line of code is written — exactly how senior engineers do it in production companies.

i Phase 1 Goal

By the end of this phase you will have:

- A fully designed 3-tier application architecture on paper (and in your head)
- A complete folder structure created on your machine
- An understanding of every component, port, and security boundary
- A foundation that every later phase builds directly on top of

Step 1 — Create Your Project Folder Structure

Open a terminal. Every command from this point forward runs inside this terminal unless told otherwise.

STEP 1 Create the root project folder

```
# Create and enter the main project folder
mkdir shopnest && cd shopnest

# Create all required subdirectories in one command
mkdir -p frontend/src/{components,pages,services}
mkdir -p frontend/public
mkdir -p backend/src/{controllers,routes,models,middleware}
mkdir -p database/{migrations,seeds}
mkdir -p terraform/{modules/{vpc,ec2,rds,sg,alb},environments/{dev,prod}}
mkdir -p ansible/{inventory,playbooks,roles/{docker,kubernetes,app}/{tasks,handlers,templates,v
ars}}
mkdir -p k8s/{frontend,backend,database}
mkdir -p helm/shopnest/{templates,charts}
mkdir -p jenkins
mkdir -p monitoring/{prometheus,grafana/{dashboards,provisioning}}
mkdir -p logging
mkdir -p .github
```

Verify the structure was created correctly:

```
find . -type d | sort
# You should see 40+ directories listed
```

STEP 2 Verify your folder structure looks like this

```
shopnest/
├── frontend/
```

```
├── public/
├── src/
│   ├── components/
│   ├── pages/
│   └── services/
├── backend/
│   └── src/
│       ├── controllers/
│       ├── routes/
│       ├── models/
│       └── middleware/
├── database/
│   ├── migrations/
│   └── seeds/
├── terraform/
│   ├── modules/
│   │   ├── vpc/
│   │   ├── ec2/
│   │   ├── rds/
│   │   ├── sg/
│   │   └── alb/
│   └── environments/
│       ├── dev/
│       └── prod/
├── ansible/
│   ├── inventory/
│   ├── playbooks/
│   └── roles/
├── k8s/
├── helm/
├── jenkins/
├── monitoring/
└── logging/
```

Step 2 — Understand the 3-Tier Architecture

Before writing any code, you must understand WHY we split into 3 tiers. This is the most important conceptual foundation of the entire project.

Tier	What it Does	Technology	Where it Runs
Tier 1: Frontend	Serves the web page the user sees and interacts with	React + Nginx	EC2 in Public Subnet
Tier 2: Backend	Processes business logic, handles API requests	Node.js + Express	EC2 in Private Subnet
Tier 3: Database	Stores and retrieves all persistent data	PostgreSQL on RDS	DB Subnet (no internet)

Step 3 — Understand the Request Flow

Trace exactly what happens when a user clicks 'Add to Cart'. This flow repeats for every user action:

i Request Flow — Add to Cart Example

1. User's browser → clicks 'Add to Cart' button on the React page
2. React app → sends POST /api/cart to the backend API
3. Nginx → receives the request on port 80, proxies /api/* to port 3000
4. Node.js API → validates the JWT auth token from the request header
5. Node.js API → writes cart item to Redis cache (fast, for session use)
6. Node.js API → writes cart item to PostgreSQL (persistent, for order history)
7. PostgreSQL → confirms the INSERT and returns success
8. Node.js API → sends back { success: true, cartCount: 3 }
9. React app → updates the cart icon count in the header
10. User's browser → sees the cart badge change from 2 to 3

Step 4 — Design the Network Architecture

AWS uses a Virtual Private Cloud (VPC) to isolate your infrastructure from the internet. Think of a VPC as your own private data centre inside AWS. Inside the VPC we create subnets — small network segments — and control who can reach what.

► 4.1 — Plan the IP Ranges

Network Component	CIDR Range	Meaning
VPC (entire network)	10.0.0.0/16	65,536 total IP addresses — the whole private network
Public Subnet A (AZ-a)	10.0.1.0/24	256 IPs — frontend EC2 and ALB live here
Public Subnet B (AZ-b)	10.0.2.0/24	256 IPs — second AZ for high availability
Private Subnet A (AZ-a)	10.0.3.0/24	256 IPs — backend EC2 lives here
Private Subnet B (AZ-b)	10.0.4.0/24	256 IPs — second AZ for backend
DB Subnet A (AZ-a)	10.0.5.0/24	256 IPs — PostgreSQL primary lives here
DB Subnet B (AZ-b)	10.0.6.0/24	256 IPs — PostgreSQL replica lives here

i Why Two Availability Zones?

AWS organises data centres into Availability Zones (AZs). If one AZ goes down (power failure, hardware fault), the other AZ keeps serving traffic. By placing resources in two AZs, we get automatic failover — a production requirement for any system with an uptime SLA.

► 4.2 — Plan the Security Groups

A Security Group is a virtual firewall around each component. It controls which IP addresses or other security groups can send traffic to it, and on which port. Rule of thumb: if two components don't NEED to talk to each other, they MUST NOT be able to.

Security Group	Inbound Allowed	Inbound Port	Why
sg-alb	Internet (0.0.0.0/0)	443 (HTTPS)	Anyone on the internet must reach the load balancer
sg-frontend	sg-alb only	80 (HTTP)	Only the ALB forwards traffic to the frontend
sg-backend	sg-frontend only	3000 (API)	Only the frontend (via proxy) calls the API
sg-redis	sg-backend only	6379	Only the backend reads/writes the cache
sg-database	sg-backend only	5432	Only the backend queries the database
sg-bastion	Your IP only (x.x.x.x/32)	22 (SSH)	Only you can SSH in — no one else

Step 5 — Plan the AWS Infrastructure Components

Document every AWS service you will create in Phase 6 (Terraform). Building this list now prevents surprises later.

AWS Service	Component Name	Purpose	Phase Created
VPC	shopnest-vpc	Isolated private network for all resources	Phase 6
Internet Gateway	shopnest-igw	Allow public subnets to reach the internet	Phase 6
NAT Gateway	shopnest-nat	Allow private subnets to reach internet (1-way)	Phase 6
ALB	shopnest-alb	Distribute incoming HTTPS traffic to frontend	Phase 6
EC2 (Frontend)	shopnest-frontend-ec2	Runs the React + Nginx container	Phase 6
EC2 (Backend)	shopnest-backend-ec2	Runs the Node.js API container	Phase 6
EC2 (Bastion)	shopnest-bastion	SSH jump server for secure access	Phase 6
RDS PostgreSQL	shopnest-db	Managed PostgreSQL with Multi-AZ	Phase 6
ElastiCache Redis	shopnest-redis	Managed Redis for caching	Phase 6
S3 Bucket	shopnest-assets	Static file storage (images, PDFs)	Phase 6

ACM Certificate	shopnest-cert	TLS/HTTPS certificate for the domain	Phase 6
Route53	shopnest.yourdomain.com	DNS routing to the ALB	Phase 6
IAM Roles	shopnest-ec2-role	Allow EC2 to access S3, ECR, etc.	Phase 6
ECR Repository	shopnest-ecr	Private Docker image registry	Phase 5

Step 6 — Create Core Project Files

Create the foundational files that every phase depends on. You will fill in the actual content in later phases, but creating the files now establishes the project skeleton.

STEP 3 Create the root .gitignore file

In your terminal, make sure you are inside the shopnest/ folder, then run:

```
cd ~/shopnest # Make sure you are in the project root

cat > .gitignore << 'EOF'
# — Node.js —————
node_modules/
npm-debug.log*
.npm

# — Environment files —————
.env
.env.local
.env*.local
*.env

# — Build output —————
build/
dist/
.next/

# — Docker —————
*.dockerignore

# — Terraform —————
**/.terraform/
*.tfstate
*.tfstate.backup
*.tfvars
!*.tfvars.example
.terraform.lock.hcl

# — Ansible —————
*.retry
ansible/vault_pass.txt

# — IDE / OS —————
.idea/
.vscode/
.DS_Store
Thumbs.db

# — Logs —————
logs/
*.log
EOF
```

STEP 4 Create the root README.md

```
cat > README.md << 'EOF'
# ShopNest DevOps Project
```

```

A full end-to-end DevOps project implementing a 3-tier e-commerce application
using industry-standard tools and AWS infrastructure.

## Architecture
- **Frontend**: React 18 + Nginx
- **Backend**: Node.js + Express REST API
- **Database**: PostgreSQL 15 on AWS RDS
- **Cache**: Redis 7 on AWS ElastiCache

## Tech Stack
Docker · Jenkins · Terraform · Ansible · Kubernetes · Prometheus · Grafana · Loki

## Phases
| Phase | Topic |
|-----|-----|
| 1 | Project Design |
| 2 | Local Development |
| 3 | Git & Version Control |
| 4 | Containerisation |
| 5 | CI/CD Pipeline |
| 6 | Infrastructure as Code |
| 7 | Configuration Management |
| 8 | Kubernetes Deployment |
| 9 | Helm |
| 10 | Monitoring & Observability |
| 11 | Logging |
| 12 | DevSecOps |
| 13 | Backup & DR |
| 14 | Production Readiness |
| 15 | Interview Prep |
EOF

```

STEP 5 Create placeholder Dockerfile files in each tier

These will be filled in Phase 4. Creating them now gives Git something to track.

```

# Create frontend Dockerfile
touch frontend/Dockerfile
touch frontend/nginx.conf
touch frontend/.dockerignore

# Create backend Dockerfile
touch backend/Dockerfile
touch backend/.dockerignore

# Create docker-compose file
touch docker-compose.yml

# Create the Jenkinsfile
touch jenkins/Jenkinsfile

# Confirm all files exist
find . -name 'Dockerfile' -o -name 'docker-compose.yml' -o -name 'Jenkinsfile' |
sort

```

Step 7 — Plan Your Port Map

Every service in this project listens on a specific port. Learn these now — you will reference them hundreds of times across all phases.

Service	Port	Protocol	Accessible From
Nginx (frontend container)	80	HTTP	ALB security group
Node.js API (backend)	3000	HTTP	Frontend security group
PostgreSQL (RDS)	5432	TCP	Backend security group only
Redis (ElastiCache)	6379	TCP	Backend security group only
SSH (Bastion host)	22	TCP	Your IP only
ALB (internet-facing)	443	HTTPS	0.0.0.0/0 (everyone)
Jenkins (CI/CD server)	8080	HTTP	Bastion or VPN only
Prometheus	9090	HTTP	Internal monitoring subnet
Grafana	3000	HTTP	Internal or via ALB
Node Exporter (metrics)	9100	HTTP	Prometheus scrape only

Step 8 — Phase 1 Validation Checklist

Before calling Phase 1 complete, verify every item below. If any item is not done, go back and complete it.

✔ Phase 1 Done When All of These Are True

- ☐ shopnest/ project folder exists with all subdirectories created
- ☐ .gitignore is created and covers Node, Terraform, Ansible, and IDE files
- ☐ README.md exists at the project root
- ☐ Placeholder Dockerfiles exist in frontend/ and backend/
- ☐ docker-compose.yml placeholder exists at project root
- ☐ jenkins/Jenkinsfile placeholder exists
- ☐ You can explain the 3-tier architecture without looking at notes
- ☐ You know which port each service listens on
- ☐ You understand why the database lives in a subnet with no internet route
- ☐ You understand what a Security Group does and why sg-database only allows sg-backend
- ☐ AWS account is created and IAM user devops-admin is set up
- ☐ All 8 tools (AWS CLI, Docker, Node.js, Git, Terraform, kubectl, Helm) are installed

COMMON MISTAKES & TROUBLESHOOTING

Every DevOps engineer hits these issues. Read this before you start, then refer back when something breaks.

Mistake 1 — Running as Root User

i The Problem

You run everything as root or use your AWS root account for daily work. If anything breaks, credentials leak, or someone gets access — everything is exposed.

✓ The Fix

On Linux: always use a regular user. Use sudo only for specific commands that need it.
On AWS: always log in as devops-admin (the IAM user), never the root account.
Check who you are: run `aws sts get-caller-identity` before doing anything in AWS.

Mistake 2 — Putting Secrets in Code

i The Problem

You paste your database password or AWS key directly into a config file, then push it to GitHub. Within minutes, bots scan GitHub and steal the credentials.

✓ The Fix

Always use .env files for secrets. The .gitignore we created in Step 3 already blocks .env from being committed.
Rule: if it is a password, key, token, or connection string — it goes in .env, never in code.
In Phase 5 (Jenkins) and Phase 12 (DevSecOps) we will add automated scanning to catch this.

Mistake 3 — Skipping the Security Group Design

i The Problem

You create a single security group that allows "all traffic from everywhere" to get things working quickly. In production this means your database is reachable from the internet.

✓ The Fix

Use the security group design from Step 4. Every rule must have a specific source (another SG or your IP).

Test it: after setting up EC2 in Phase 6, try to connect directly to port 5432 from your laptop. It should time out.

If it connects, your security group is wrong. Fix it before moving on.

Mistake 4 — Not Using Multiple Availability Zones

i The Problem

You deploy everything into a single AZ to save money. The AZ has a partial outage. Your entire application goes down.

✓ The Fix

Always use at least 2 AZs for every tier. For this project: ap-south-1a and ap-south-1b.

RDS Multi-AZ standby is in a different AZ so failover is automatic.

The cost difference for a learning project is under \$5/month. It is worth it for the practice.

Mistake 5 — Forgetting the NAT Gateway

i The Problem

Your backend EC2 is in a private subnet. When Ansible runs `apt install docker.io` on it, the command hangs forever because the server has no route to the internet.

✓ The Fix

The NAT Gateway in the public subnet gives private-subnet instances ONE-WAY internet access.

Private instances can initiate outbound connections (to download packages, call external APIs).

But inbound connections from the internet cannot reach them — that is the security benefit.

Verify: from the backend EC2 (via bastion SSH), run `curl https://google.com`. If it works, NAT is set up correctly.

WHAT COMES NEXT — PHASE 2 PREVIEW

Phase 1 is complete. You now have a fully designed architecture and a project skeleton. In Phase 2 we go hands-on with actual code.

i Phase 2 — Local Development Setup

In Phase 2 you will:

- Write the React frontend application from scratch
- Write the Node.js REST API from scratch
- Set up a local PostgreSQL database with schema migrations
- Configure Redis for session and cart caching
- Create .env files for all environment variables
- Run all 3 tiers locally and verify they talk to each other
- Test every API endpoint using curl and Postman
- Add health check endpoints to frontend and backend

When Phase 2 is done you will have a working ShopNest application running entirely on your laptop — no cloud needed yet.

Quick Reference — Phase 1 Key Concepts

Keep this page open while working on later phases. These facts do not change.

Concept	Value / Detail
Project name	shopnest
AWS region	ap-south-1 (Mumbai) — change to your nearest region
VPC CIDR	10.0.0.0/16
Frontend port	80 (Nginx inside container)
Backend port	3000 (Node.js)
Database port	5432 (PostgreSQL)
Cache port	6379 (Redis)
Jenkins port	8080
Prometheus port	9090
Grafana port	3000 (separate from backend — different server)
Database name	shopnest_db
Database user	shopnest_user
IAM user for DevOps	devops-admin
ECR repo name	shopnest

EKS cluster name	shopnest-cluster
Helm release name	shopnest
Kubernetes namespace	shopnest

ShopNest DevOps Project — Phase 1 | Reply 'next' to receive Phase 2